Online Closed-Loop Agentic Fuzzing and Adaptive Feedback Loop

Guidance Loop: Mutation Hints → Generator

Orchestration & Seed Synthesis

Deterministic Orchestrator

Fixed Budget T

- Locks Target Invariant ϕ_i+ Dedicated ISA Checker
- Allocates uniform execution budget T per target
- Ensures equal opportunity across all invariants

Target Queue: Invariant Catalog Φ (topological dispatch)

LLM Generator Agent

Checker-Aware

Synthesizes C/C++ Seeds ('seed_sp_probe.c')

Conditioned on formal ϕ_i+ Level-3 predicates from Stage 3

Anti-Dead Code Elimination (Anti-DCE) Design

1. Dynamic VLAs (argc bounds):

```
int n = argc > 1 ? atoi(argv[1]) : 64;
char buf[n];
// runtime dimensions force dynamic stack frame
```

2. Escaping Volatile Barriers:

```
asm volatile("" : "+m"(buf) : : "memory");
volatile char sink = buf[0];
// prevents middle-end SRA / DCE elimination
// forces compiler backend to emit defense check
```

3. External Non-Inlinable Calls:

```
__attribute__((noinline))
void trigger_guard(char *p);
// prevents inlining; preserves complete prologue,
// return address LR', and epilogue stack check
```

Seed .c

Instrumented Build & Dual Oracle

Rebuilt Compiler Driver (GCC_INST)

Standalone Binary

GNU GCC 16.1 | Clang / LLVM 21 | x86_64, AArch64, RV64

Compilation Flags & Defense Profiles:

```
-fstack-protector-strong -fstack-clash-protection
-fsanitize=safe-stack -mbranch-protection=standard
```

GCC_INST Execution Stream (UID Trace)

```
[DEFUZZ_LOG:SP#a3f2:hit] expand_stack_protection()
has_vla=1, stack_protect_decl=1, canary_slot=0x30
[DEFUZZ_LOG:SP#b109:hit] aarch64_layout_frame()
LR_offset=0x20, canary_offset=0x38 (ALERT: LR < canary!)
[DEFUZZ_BIN] Compiled Target Binary: ./a.out (deterministic)
>> Streamed directly to Dual Deterministic Oracle
```

Dual Deterministic Machine Oracle O(P, I, S)

Deterministic Function: O : P × I × S → {PASS, FAIL, NA, ERROR}

Static Audit Checker

- GIMPLE / LLVM IR Audit
  Verifies __stack_chk_fail call
- DWARF Frame Inspector
  Extracts stack layout offsets
- Assembly Disassembler
  AArch64 / x86 prologue check

Evaluates unexecuted binaries
Zero-execution static audit

Dynamic Sandbox Probe

- Isolated Execution Box
  QEMU / Bare-metal execution
- Runtime Overflow Trigger
  Executes off-by-one / VLA probe
- Trap & Signal Handler
  SIGSEGV vs Canary exit(0)

Observes live hardware traps
Runtime verification proof

Verdict

Verdict Dispatch & Closed Loops

Feedback Agent (Advanced Multimodal Analysis)

Multi-Dimensional Feedback Ingestion via Bash & MCP:

- Compiler diagnostics, warnings, and optimization records (-fopt-info)
- UID probe hits: `hit_count(UID)` tracking unreached critical conditions
- Detailed Oracle execution traces and DWARF register offsets

1. **Guidance Loop (Mutation Hints)**
   If condition unreached (e.g., has_vla == false), generates precise syntactic mutation instructions to steer Generator Agent.

2. **Checker Adaptive Rewrite (Evolution & Pool Replay)**
   When rule ambiguity votes ≥ τ: triggers automated checker rewrite, **AND immediately replays the entire historical seed pool H** against new checker to catch previously masked violations.

Rewrite & Replay Pool H

NO: PASS / NA / ERROR

Invariant Violated?

YES: FAIL

CONFIRMED BUG

Deterministic Violation Proven!
Zero-false-positive binary trace

Automated C-Reduce
Delta-debugging + semantic test

Output: Minimal Zero-Day PoC

3. Lateral Exploration Loop (Bug Amplification Mechanism)

Does NOT terminate on first bug! Drives Generator to amplify finding:

- Mutate Boundary Types: signed/unsigned wraps, multi-dimension VLAs
- Composite Structures: nested structs, flexible array members, alloca() chains
- Atypical Stack Alignment: forced 32/64-byte vector alignment, SVE registers

Maximal Vulnerability Discovery across Alternate Backend Lowering Paths

Lateral Exploration Loop: Bug Amplification (Variants)